

OT ネットワークにおける、インライン型セキュリティブリッジの動作実験

概要

OT ネットワークでは可用性が重視される。また、ネットワーク構成を変更することが困難な場合が多い。制御システムに対して異常値を書き込む Modbus 通信を遮断する産業用 FW として、ネットワークの外側に配置されたプロミスキヤスモードで行うものは存在するが、インライン配置にすることで攻撃を検知し、その場で遮断することでサイバー攻撃の被害を抑えることが可能となる。産業制御シミュレータ GRFICSv3 の化学プラントに対する異常値を観察し、ルールベースの異常検知を使用したインライン型セキュリティブリッジを実装し、その動作実験を行った。セキュリティブリッジは bridge.py、攻撃コードは attack_demo_10.py であり、どちらも Kali Linux で実行した。その PLC・バルブ・Kali は 192.168.95.0/24 のネットワーク内のノードとなるように IP アドレスの設定を行った。PLC は 192.168.95.2、Kali は 192.168.95.111、バルブは 192.168.95.10～13 まで設定した。簡単のため、PLC から 192.168.95.10 のバルブ間の Modbus 通信のみを考えた。ARP スプーフィングを用いて PLC とバルブの間に Kali を配置し、暫定的にインライン配置を再現した。Kali の IP テーブルの設定を行った。Modbus 通信で使用するポート番号 502 のパケットをバルブに転送する前に一度 Kali が受け取るように変更した。bridge.py を実行後、attack_demo_10.py を実行するとターミナルに[BLOCKED]というログが表示され、許可された以外の IP アドレスからの書き込み命令を遮断したことが確認できた。今回のように、GRFICSv3 の化学プラントに特化した閾値を定めるためには、制御システムを十分に観察することで可能だが、実際の OT 環境では制御システムごとに異常値と判断するための閾値が異なるため、ルールベースの異常検知では限界がある。今後は、正常の通信から統計的に判断し、閾値をブリッジ自身で設定するようなものを実装することを目指す。

1. 目的

OT ネットワークでは、IT とは異なり可用性が最も重視されている。ネットワーク構成を変更するのも困難な環境で、制御システムに対する攻撃のパケットのみを遮断するインライン型セキュリティブリッジの実装を行なった。産業制御シミュレータ上で実際に攻撃コードを実行し、パケットを遮断できるかどうかの確認を目的とした。

2. 実験方法

2.1 実験環境

Windows11 マシン, VirtualBox, Kali Linux, 産業制御シミュレータ GRFICSv3, Wireshark

2.2 実験手順

GitHub「<https://github.com/mrideout/GRFICSv3>」から Simulation VM, PLC VM の ova ファイルをそれぞれダウンロードした。Windows マシンで VirtualBox を起動し、ova ファイルから VM を作成した。次に、「<https://www.kali.org/get-kali/#kali-platforms>」で Kali Linux のイメージをダウンロードし、VM を作成した。ネットワークの設定を行い、作成した VM 全てを 192.168.95.0/24 のノードとした。Windows マシンでブラウザを開き、192.168.95.10 と入力すると、GRFICSv3 の HMI 画面を確認することができる(図 1)。Simulation VM は PLC の制御先である 4 個のバルブであり、それぞれの IP アドレスは次の表 1 に示す。バルブの位置は図 1 に対応している。

表 1 それぞれのバルブの IP アドレスと位置の対応

IP アドレス	バルブの位置
192.168.95.10	左上
192.168.95.11	左下
192.168.95.12	右上
192.168.95.13	右下



図 1 GRFICSv3 の HMI 画面

左側のバルブはリアクターに A,B を投入する際の管の開度を調節している。また、右側のバルブはリアクターから出力される蒸気や製造物を通る管の開度を調節している。PLC VM は PLC であり、IP アドレスは 192.168.95.2 と設定されている。PLC とバルブ間では Modbus TCP によるマスタ・スレーブ方式の通信が行われており、リアクター内部の圧力を制御している。Kali Linux の IP アドレスは 192.168.95.111 とした。本実験は簡単のため、PLC と左上のバルブの間のみに Kali をインライン接続する。以上より、ネットワーク構成図を図 2 に示す。

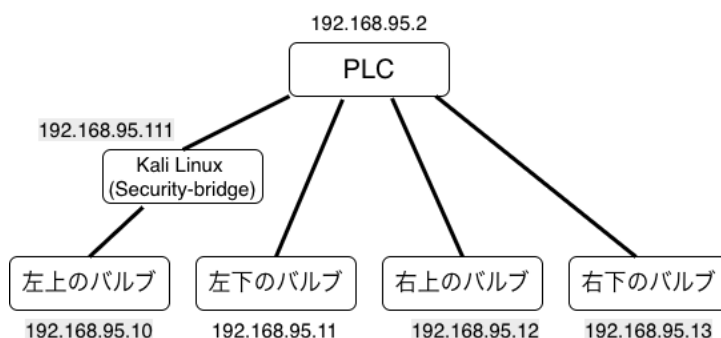


図 2 ネットワーク構成図 (192.168.95.0/24)

次に Kali の IP フォワードが 0 であることを確認した。IP フォワードが 1 の場合、PLC からのパケットを OS がセキュリティブリッジを介さずにバルブへ転送してしまうからである。GRFICSv3 の仕様上、PLC とバルブ間にセキュリティブリッジをインライン接続するのは困難であった。ARP スプーフィングを行うことで全てのパケットを Kali 経由で転送すればインラインを暫定的に再現できると考えた。また、Kali で IP テーブルの設定を行った。通常、セキュリティブリッジのコードを実行するだけでは、PLC からバルブへの Modbus TCP パケットの IP ヘッダーの宛先 IP アドレスが 192.168.95.10 であるため Kali は自分宛ではないと判断し、無視してしまう。Kali Linux の IP テーブル機能を使うことで、Modbus TCP 通信のみを一度受け取るように設定を変更することが可能となる。Kali のターミナルに「`sudo iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 502 -j REDIRECT --to-port 502`」と入力する。`-A PREROUTING` はパケットが届いた直後にこの命令を適用することを意味している。`-p tcp -dport 502` は TCP プロトコルの宛先ポート番号が 502 であることを示している。PLC からバルブへの Modbus TCP は宛先ポート番号が 502 で固定である。以上の設定を図 3 に示す。

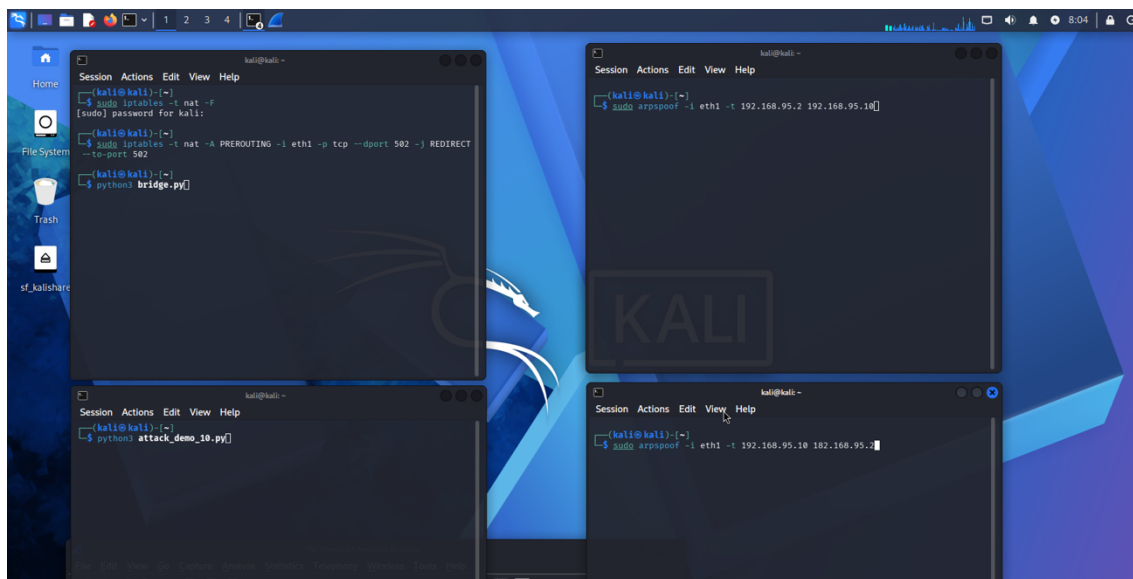


図 3 環境構築のためのコマンド群

次に、`bridge.py` と `attack_demo_10.py` を図 4、図 5 に示す。まず、`socket` ライブラリと `threading` ライブラリをインポートする。`inspect` 関数では、フィルタリングルールを定義している。今回は送信元 IP アドレスが `ALLOW_IPS` というリストに含まれないパケットは遮断している。`ALLOW_IPS` には PLC の IP アドレスのみが格納されている。また、Modbus ヘッダーのファンクションコード(以下 FC)が 6 のパケットのうち、60000 以上の値や 0 を書き込む命令は遮断している。PLC やバルブにはデバイスメモリが存在する。デバイスメモリにはコイル・離散入力・入力レジスタ・保持レジスタがある。1 アドレスにつき 1bit

の値を格納できるコイルや離散入力をビットデバイス、1 アドレスにつき 16bit の値を格納できる入力レジスタや保持レジスタをワードデバイスと呼ぶ。離散入力や入力レジスタは読み取り専用であるが、コイルや保持レジスタは PLC からの Modbus 通信によって書き込みが可能である。FC6 の Modbus 通信は保持レジスタの値を書き込む命令であるため、遮断している。GRFICSv3 では正常時は FC が 4 の入力レジスタ読み取り命令と、FC16 の保持レジスタ書き込み命令のみを行っている。attack_demo_10.py を実行することで生成されるパケットは FC6 であるため、本実験では FC6 のみを遮断すればセキュリティブリッジの動作確認としては十分である。遮断した際はログを出力している。また、FC16 のパケットは全て通していることも確認できる。

```
# 研究目的のみ、ホストオンリーの環境でのみ使用

import socket
import threading

LISTEN_HOST = '0.0.0.0'          # どの NIC でも OK という意味, 要は使用する NIC の IP アドレス
LISTEN_PORT = 502
VALVE_HOST = '192.168.95.10'    # どこに connect するかで使用するからバルブの IP アドレス
VALVE_PORT = 502

# 正常な送信元 IP のリスト, PLC の IP アドレス
ALLOWED_IPS = ['192.168.95.2']

# inspect 関数
def inspect(data, src_ip):
    # パケットが短すぎる場合は通す
    if len(data) < 8:
        return True

    # 1. 送信元 IP チェック, Attack_demonstration.py は送信元が kali の IP アドレスなのでこれで遮断できる
    if src_ip not in ALLOWED_IPS:
        print(f'[BLOCKED] Unauthorized IP: {src_ip}')
```

```

        return False

    fc = data[7] # ファンクションコード, Wireshark の Modbus TCP 以上の
    値が data に格納される 右下の画面と対応を考えればわかる

    # 2. FC6 (Single Register Write) のチェック
    if fc == 6:
        value = int.from_bytes(data[10:12], 'big')
        if value >= 60000 or value == 0:
            print(f'[BLOCKED] FC6 abnormal value: {value}')
            return False
        # 正常値なら許可
        print(f'[ALLOW] FC6 value: {value}')
        return True

    # 3. FC16 (Multiple Registers Write) のチェック
    elif fc == 16:
        print(f'[ALLOW] FC16 detected from {src_ip}')
        return True

    # 4. FC4 など上記以外のパケット
    else:
        return True

    return True

def handle_connection(conn, addr):
    src_ip = addr[0]

    # 誰が接続してきたかのログを標準出力する
    print(f'[INFO] Connected from {src_ip}')

    # バルブへの接続, try-except は例外処理, except 例外型: 例外型と同じ例
    外発生時の処理
    try:

```

```

valve_sock = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
valve_sock.connect((VALVE_HOST, VALVE_PORT))

# Exception は ZeroDivisionError 等のあらゆる例外型を総括した例外
型, e と一旦名付ける必要がある
except Exception as e:
    print(f'[ERROR] Cannot connect to Valve: {e}')
    # 例外が発生したらスレッドを終了する
    conn.close()
    return

try:
    while True:
        # 4096byte のデータを data に格納, ページサイズが 4096 だから 2 の
        冪乗をよく使う
        data = conn.recv(4096)

        # data が空だったら通信を終わる
        if not data:
            break

        print(f'[RCV] {data.hex()}')

        if not inspect(data, src_ip):
            # パケットを捨てて次を待つ (接続は維持), inspect 関数の戻り値
            が False なら if 文を実行, Valve へ転送しない
            continue

        # Valve へ転送
        valve_sock.sendall(data)

        # Valve からのレスポンスを受け取るまで待ち状態 その後、PLC へ返す
        response = valve_sock.recv(4096)
        conn.sendall(response)

```

```

except Exception as e:
    print(f'[ERROR] {e}')
finally:
    conn.close()
    valve_sock.close()
    print(f'[INFO] Connection closed: {src_ip}')

def main():
    # AF_INET : IPv4 プロトコルファミリー, SOCK_STREAM : TCP ソケット
    server_sock = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)

    # Kali のどの NIC(0.0.0.0)に届いたものでも ok, 宛先ポート番号 502 のパケ
ットならね
    server_sock.bind((LISTEN_HOST, LISTEN_PORT))

    # ソケットを接続待ち状態にする
    server_sock.listen(5)

    #
    print(f'[INFO] Bridge listening on port {LISTEN_PORT}')
    print(f'[INFO] Forwarding to Valve: {VALVE_HOST}:{VALVE_PORT}')

    while True:
        # 通信が来るまでブロッキング, conn に新しいソケットを, addr に送信元
        (PLC)IP アドレスを格納
        conn, addr = server_sock.accept()

        # 接続ごとに分身(スレッド)を作って処理する, Thread メソッドの第一引数
        はやらせたい仕事を,
        # 第二引数は handle_connection 関数に必要な 2 個の引数をタプルで渡す
        thread = threading.Thread(target=handle_connection,
args=(conn, addr))

```

```

        # 本体が終了したら一緒に死んでいいよ, start()メソッドでスレッドが別の
        時間軸で動き出す
        thread.daemon = True
        thread.start()

if __name__ == '__main__':
    main()

```

図 4 bridge.py のソースコード

FC6 で異常値を書き込む命令のパケットを生成した。今回は、外部ライブラリ pymodbus を使用した。3 行目で IP アドレスが 192.168.95.10 ではなく、192.168.95.111 なのは理由がある。バルブの IP アドレスにして実行するとパケットがブリッジに届かないという問題が発生した。これは、OS 自身が自分宛でないパケットを遮断してしまうからだと判断し、本来の実験環境とは少しずれてしまうが、自身の IP アドレスとすることでブリッジへパケットを送ることを優先し、実験を続けた。while ブロック内の 2 行目はバルブ(192.168.95.10)の保持レジスタのアドレス 1 番を 65535 に書き換える命令を意味している。FC16 は保持レジスタのアドレス値を複数点書き換えるのに対し、FC6 は一点のみを書き換える。また、一点とは、1 アドレスに格納される bit 数のことで、保持レジスタに限れば 16bit のことである。

```

from pymodbus.client import ModbusTcpClient

client10 = ModbusTcpClient('192.168.95.111')

try:
    while True:
        # 第一引数は書き込み先のレジスタ番号, 第二引数は書き込む値
        res = client10.write_register(1, 65535)
        print("10:", res)

# KeyboardInterrupt は例外型の 1 つ, Exception の子ではない
except KeyboardInterrupt:
    client10.close()

```

図 5 attack_demo_10.py のソースコード

bridge.py を実行した後に、attack_demo_10.py を実行すると図 6 が標準出力された。最初は、[INFO] Connected from 192.168.95.2 とあるため、PLC からのパケットがセキュリティブリッジに届いていることがわかる。[RECV]でパケットの中身が 16 進数で表示されている。ここでは TCP ヘッダーより上、すなわち Modbus TCP ヘッダーと Modbus ヘッダーの値が表示されている。その後、ログが出力されることなく再び[RECV]が続いているため、inspect 関数の条件分岐に加えなかった FC4 のパケットであることが推測できる。[ALLOW] FC16 detected from 192.168.95.2 とあることから、PLC からの FC16 のパケットをブリッジが通したことが確認できる。[INFO] Connected from 192.168.95.111 は attack_demo_10.py を実行し、生成したパケットがブリッジと接続を行ったことを意味している。[BLOCKED] Unauthorized IP: 192.168.95.11 という出力結果からもわかるように、セキュリティブリッジが ALLOW_IPS に含まれない送信元 IP アドレス(192.168.95.111)からのパケットを遮断していることが確認できた。



本実験では、簡易的なセキュリティブリッジの動作確認を行った。inspect 関数ではルールベースの異常検知を行っており、内容も GRFICSv3 の化学プラントに特化している。実際の OT ネットワークでは、制御システムごとに異常値の閾値が異なるため、システム自体をよく観察することで閾値を定める必要があるが、それでは汎用的なセキュリティブリッジを開発することはできない。統計的な異常検知を行うことで、ブリッジ自らが閾値を定め、正常値からズレた値の書き込み命令を遮断することが今後の目標である。

参考文献

福田 敏博,「現場で役立つ OT の仕組みとセキュリティ 演習で学ぶ! わかる! リスク分析と対策」, 翔泳社, 2021